

Introdução a Padrões de Projeto: O Papel do GoF e os 23 Padrões Clássicos

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

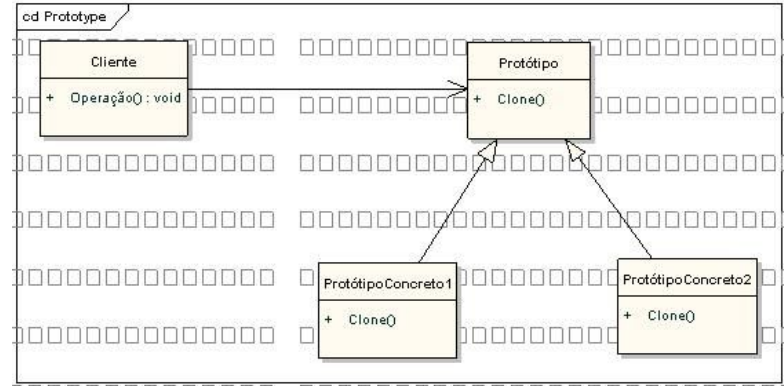
- Motivação e Conceito Fundamental
- O que são Padrões de Software
- Por que usar padrões de software
- Áreas e Tipos de Padrões
- Um Pouco da História
- Estrutura de um Padrão
- Formatos de Descrição de Padrões
- Padrões e o Reuso de Software
- Classificação GoF — A Essência dos 23 Padrões
- Padrões Creacionais
- Padrões Estruturais
- Padrões Comportamentais
- Benefícios dos Padrões GoF
- Cuidados e Limitações

Motivação e Conceito Fundamental

- No desenvolvimento de software, certos problemas aparecem repetidamente.
- Em vez de reinventar a roda, podemos reutilizar soluções já testadas e aprovadas. Essa ideia dá origem ao conceito de padrão de software.
- **Definição (Coplien, 1994):** “Um padrão é uma solução para um problema em um contexto.”
- **Definição (Alexander, 1977):** “Um padrão descreve um problema recorrente e a essência de uma solução que pode ser usada milhões de vezes, sem nunca ser igual.”

Três elementos sempre presentes

- **Problema** (o que precisa ser resolvido)
- **Contexto** (onde e quando ocorre)
- **Solução** (abordagem reutilizável que resolve o problema)



O que são Padrões de Software

- São modelos de solução testados e reutilizáveis.
- Capturam boas práticas de design orientado a objetos.
- Promovem reuso de conhecimento, não apenas de código.
- Criam um vocabulário comum entre desenvolvedores.
- “Design Patterns são a cristalização da experiência de projetistas experientes.” — Rossana Andrade

Por que usar padrões de software

- Usar padrões de software é importante porque eles fornecem soluções testadas e reutilizáveis para problemas comuns de desenvolvimento, melhorando a eficiência, a manutenibilidade e a colaboração.
- Eles criam uma linguagem comum para a equipe, facilitam a compreensão do código, promovem a reutilização de componentes e ajudam a construir sistemas mais robustos.



Benefícios dos Padrões

- **Melhoria da comunicação:** uso de uma linguagem comum.
 - **Produtividade:** solução já validada → menor retrabalho.
 - **Reuso:** evita reinventar a roda.
 - **Modularidade:** divide o sistema em partes independentes.
 - **Manutenção facilitada:** código mais claro e evolutivo.
-
- Quando bem aplicados desde o início, reduzem retrabalho nas fases finais do projeto.

Possíveis Desvantagens (Coplien, 2000)

- **Menor eficiência:** padrões podem adicionar classes e camadas extras.
- **Menor legibilidade:** uso indevido pode tornar o código mais complexo.
- **Curva de aprendizado:** exigem conhecimento prévio de OO e boas práticas.
- Use padrões com propósito, não por moda.

Áreas e Tipos de Padrões

Tipo	Descrição	Referência
Arquiteturais	Estrutura geral do sistema	<i>Buschmann et al., 1996</i>
De Projeto	Relações entre classes e objetos	<i>Gamma et al., 1995</i>
De Análise	Soluções conceituais	<i>Fowler, 1997</i>
De Programação (Idiomas)	Específicos de linguagem	<i>Coplien, 1991</i>
De Processo / Organizacionais	Melhoria do processo e times	<i>Coplien & Harrison, 2004</i>
Anti-padrões	Más soluções e como evitá-las	<i>Brown et al., 1998</i>

Padrões Arquiteturais

- Definem a estrutura de alto nível de um sistema de software, especificando como os componentes principais se organizam e interagem. Focam na visão global da aplicação — camadas, módulos, fluxos de dados e dependências.
- **Objetivo:** Estabelecer uma macroestrutura reutilizável que possa servir de base para sistemas similares, garantindo escalabilidade, desempenho e manutenibilidade.
- **Exemplos:**
 - **Camadas (Layers):** separa responsabilidades em níveis (apresentação, domínio, persistência).
 - **Cliente-Servidor:** divide o sistema em consumidores e provedores de serviços.
 - **Microservices:** organiza o sistema como um conjunto de serviços independentes.
 - **Pipes and Filters:** processamento em cadeia de transformações.
- **Importância:** Ajudam a guiar o design inicial e a comunicação entre arquitetos e desenvolvedores, garantindo coerência estrutural.

Padrões de Projeto (Design Patterns)

- Oferecem soluções recorrentes para problemas de design orientado a objetos. Focam em como classes e objetos colaboram para resolver problemas comuns de flexibilidade e reuso.
- **Categorias:**
 - Criacionais: tratam da criação de objetos (ex: Singleton, Factory Method, Builder).
 - Estruturais: lidam com composição e organização de classes/objetos (ex: Adapter, Composite, Decorator).
 - Comportamentais: descrevem interações e responsabilidades (ex: Observer, Strategy, Command).
- **Importância:**
 - Reduzem o acoplamento, promovem reuso e tornam o código mais compreensível.
 - São a “linguagem comum” entre desenvolvedores experientes.

Padrões de Análise

- Soluções conceituais e de domínio que surgem antes da implementação, ainda na fase de modelagem e requisitos. Representam ideias recorrentes em problemas de negócio.
- Objetivo: Fornecer modelos de alto nível que podem ser reutilizados em diferentes sistemas dentro de um mesmo domínio (ex: financeiro, educacional, saúde).
- **Exemplos:**
 - Conta Bancária, Fatura, Pedido, Catálogo de Produtos — são abstrações genéricas reutilizáveis.
 - Party Pattern: generaliza a ideia de “pessoa ou organização” em sistemas de negócio.
- **Importância:** Facilitam o entendimento comum entre analistas e desenvolvedores, e conectam análise de requisitos ao design orientado a objetos.

Padrões de Programação (Idiomas)

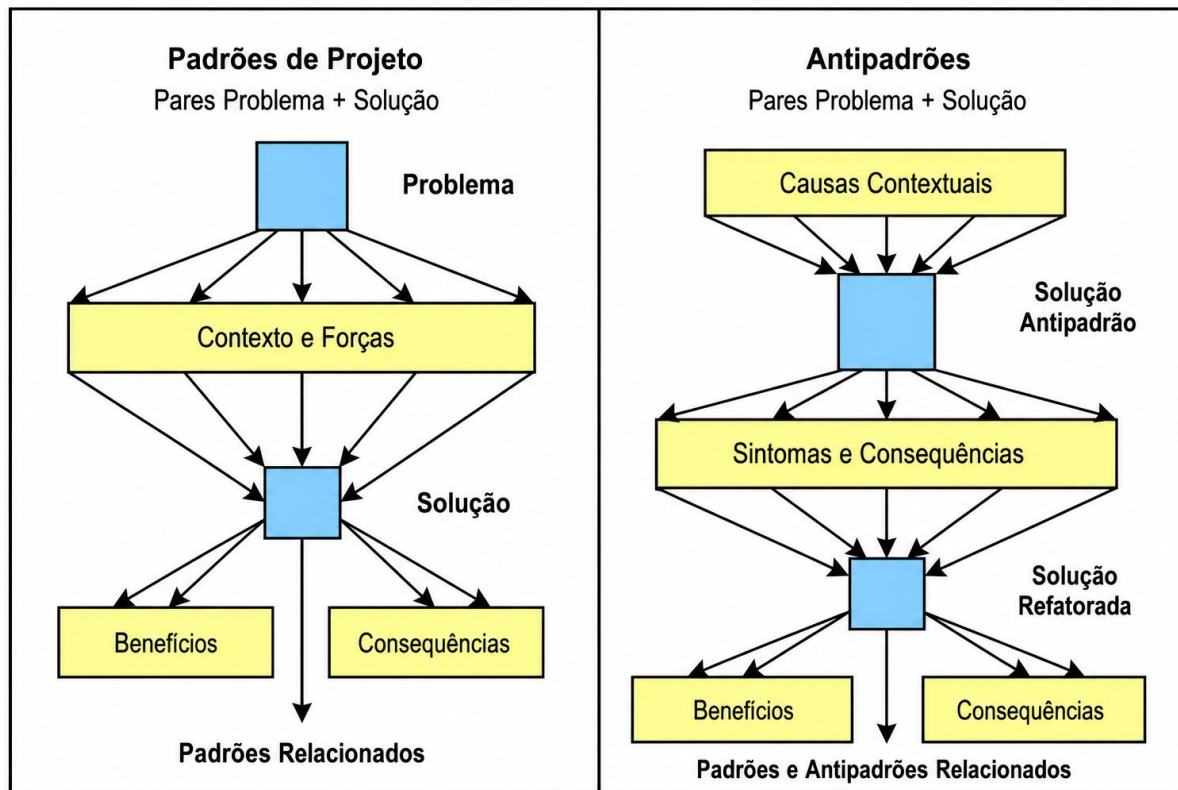
- Capturam boas práticas específicas de uma linguagem de programação. São soluções típicas ao nível de código e expressões sintáticas.
- **Objetivo:** Ajudar o desenvolvedor a tirar o máximo proveito dos recursos da linguagem, evitando armadilhas e reforçando estilo e legibilidade.
- **Exemplos:**
 - Em C++, o uso de RAI (Resource Acquisition Is Initialization).
 - Em Python, uso de context managers (with open(...) as f:).
 - Em JavaScript, padrão Module para encapsular variáveis.
 - Em Kotlin, uso de extension functions e data classes.
- **Importância:** Promovem um estilo consistente e idiomático, o que melhora a manutenção e reduz erros sutis.

Padrões de Processo / Organizacionais

- Relacionam-se a boas práticas de equipe, cultura e processos de desenvolvimento. Focam na colaboração, comunicação e estrutura organizacional.
- **Objetivo:** Melhorar a produtividade, reduzir riscos e reforçar práticas saudáveis de desenvolvimento.
- **Exemplos:**
 - Pair Programming (Programação em Pares)
 - Daily Stand-Up (Reunião Diária)
 - Architect Also Implements (o arquiteto também codifica)
 - Collective Code Ownership (código é de todos)
- **Importância:** Esses padrões sustentam o sucesso técnico por meio de práticas humanas e culturais.

Anti-Padrões

- Documentam soluções ruins ou ineficientes que parecem boas à primeira vista, mas trazem consequências negativas a longo prazo.
- **Objetivo:** Ajudar equipes a identificar e evitar erros recorrentes de design, arquitetura ou gestão.
- **Exemplos:**
 - God Object: classe que faz tudo.
 - Spaghetti Code: código confuso e sem modularidade.
 - Golden Hammer: usar sempre a mesma tecnologia para qualquer problema.
 - Copy-and-Paste Programming: duplicar código em vez de reutilizar.
- **Importância:** Contribuem para uma cultura de code review e aprendizado coletivo, ajudando a reconhecer “cheiros ruins” (code smells).



ANDRADE, Roberto; TORRES, Jenny; ORTIZ-GARCÉS, Iván. Enhancing security in software design patterns and antipatterns: a framework for LLM-based detection. **Electronics**, v. 14, n. 3, p. 586, 2025.

Um Pouco da História

- **Anos 70:** Christopher Alexander (arquitetura civil) propõe “A Pattern Language” — padrões como elementos que constroem sistemas complexos (cidades, edifícios, softwares).
- **1987:** Kent Beck e Ward Cunningham aplicam a ideia a interfaces gráficas em OO.
- **1988:** James Coplien define idioms para C++.
- **1994:** Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides publicam o livro “Design Patterns: Elements of Reusable Object-Oriented Software” — o marco do movimento GoF.
- **1993–atual:** Criação das conferências PLoP (Pattern Languages of Programs) pelo grupo Hillside.

Eventos importantes

- **1993** → Criação do grupo Hillside e das conferências PLoP (Pattern Languages of Programs).
- **1994** → Lançamento do livro do GoF no OOPSLA.
- Padrões passam a ser reconhecidos como linguagem de reuso de conhecimento na engenharia de software.

Estrutura de um Padrão

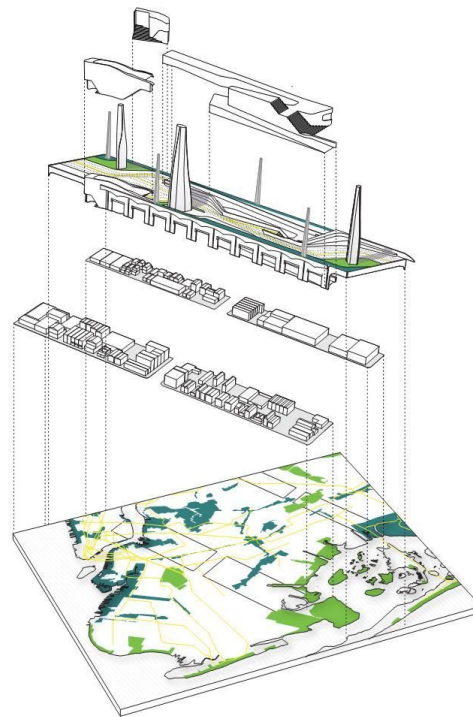
- Segundo Coplien e Alexander, um bom padrão possui:

Componente	Descrição
Nome	Identifica e dá vocabulário (ex: Singleton, Bridge).
Contexto	Situação em que o problema ocorre.
Problema	O desafio ou conflito a resolver.
Forças	Aspectos que tornam o problema difícil (ex: desempenho, acoplamento).
Solução	Estrutura ou abordagem reutilizável.
Contexto resultante	Como o sistema fica depois da aplicação.
Padrões relacionados	Outros padrões que podem complementar.

“Um padrão é uma pequena estória: tem cenário, conflito, moral e consequências.”

Estrutura narrativa de um padrão

- Um padrão é como uma pequena narrativa técnica.
- Ele não é apenas um algoritmo ou diagrama, mas uma história de design que pode ser contada, compreendida e reaplicada.



Estrutura narrativa de um padrão

Elemento narrativo	Papel no padrão	Exemplo prático
Cenário	O contexto onde o problema aparece. Define o ambiente e as restrições.	“Você está desenvolvendo um sistema com múltiplas janelas que precisam reagir a mudanças de dados em tempo real.”
Conflito	O problema recorrente que causa tensão. Mostra o dilema que o desenvolvedor enfrenta.	“Como notificar várias janelas ao mesmo tempo sem acoplar todas diretamente ao modelo de dados?”
Moral	A solução essencial — o “insight” do padrão. É a recomendação que resolve o conflito.	“Use o padrão <i>Observer</i> , permitindo que objetos se inscrevam e sejam notificados automaticamente.”
Consequências	Os efeitos e trade-offs da aplicação do padrão. Mostra o que melhora e o que pode piorar.	“Facilita a atualização dinâmica, mas pode gerar notificações redundantes e custos de desempenho.”

Formatos de Descrição de Padrões

- Os padrões não são apenas ideias ou nomes de soluções — eles precisam ser documentados de forma que outras pessoas possam compreendê-los e reutilizá-los.
- Ao longo do tempo, surgiram três grandes formatos de documentação, cada um com um estilo e propósito diferentes.

Formato GoF (Gang of Four)

- **Referência:** Gamma, Helm, Johnson, Vlissides — Design Patterns: Elements of Reusable Object-Oriented Software (1994)

- **Características**
 - É o formato mais técnico e estruturado, amplamente usado na engenharia de software.
 - Descreve o padrão como uma ficha técnica detalhada, com seções padronizadas.
 - Busca facilitar a implementação e reuso prático em código orientado a objetos.

Estrutura típica

Seção	Função
Nome	Identificação única do padrão.
Intenção (Intent)	O que o padrão faz — o problema central que resolve.
Motivação (Motivation)	Contexto e cenário que originaram o padrão.
Aplicabilidade	Situações onde o padrão é mais útil.
Estrutura	Diagramas UML e relações entre classes/objetos.
Participantes	Quais classes e papéis estão envolvidos.
Colaborações	Como os participantes interagem.
Consequências	Vantagens, desvantagens e impactos do uso.
Implementação	Dicas práticas, exemplos de código, variações.

Exemplo simplificado — Observer

- **Intenção:** Definir uma dependência um-para-muitos entre objetos, para que, quando um objeto mudar de estado, todos os seus dependentes sejam notificados.
- **Motivação:** Manter sincronização entre modelo e interface em um editor gráfico.
- **Consequência:** Baixo acoplamento, mas aumento da complexidade de notificações.
- **Quando usar esse formato**
 - Em catálogos técnicos ou materiais didáticos de engenharia de software.
 - Quando o público precisa implementar o padrão em código.

Formato Alexandriano

- **Referência:** Christopher Alexander — A Pattern Language: Towns, Buildings, Construction (1977)
- **Características**
 - É o formato original de onde veio a ideia de “padrão”.
 - Tem uma linguagem narrativa e orgânica, próxima de um texto literário.
 - O foco está na compreensão humana e conceitual, não na codificação.
 - Usado inicialmente na arquitetura civil (edifícios, cidades) e depois adaptado ao software.

Estrutura típica

Elemento	Descrição
Nome do padrão	Representa a essência da solução.
Contexto	Situação ou ambiente onde o problema ocorre.
Problema	O conflito ou desafio a ser resolvido.
Forças	Tensão entre requisitos e restrições.
Solução	Descrição narrativa da forma de resolver o problema.
Portanto (Therefore)	Conclusão que conecta o problema à solução (“therefore...”).
Diagramas ou esboços	Representações conceituais (não necessariamente UML).

Exemplo simplificado — Observer, estilo Alexandriano

- **Contexto:** Em sistemas interativos, dados mudam frequentemente e precisam ser refletidos em múltiplas interfaces.
- **Problema:** Como manter a consistência sem criar dependências rígidas?
- **Forças:** Flexibilidade versus desempenho; simplicidade versus desacoplamento.
- **Solução:** Permita que os objetos interessados se registrem para receber notificações automáticas de mudanças.
- **Portanto:** Quando o estado do objeto central muda, todos os observadores são atualizados — mantendo o sistema coeso, porém desacoplado.
- **Quando usar esse formato**
 - Em descrições conceituais e ensino introdutório sobre o raciocínio de padrões.
 - Em padrões de análise ou organizacionais, onde o foco é humano, não técnico.

Formato Coplien (Contextual e Rationale-Based)

- **Referência:** James O. Coplien — Advanced C++ Programming Styles and Idioms (1991) e Organizational Patterns (1994)
- **Características**
 - Dá ênfase ao contexto e às forças — ou seja, ao porquê de o padrão existir.
 - Inclui uma explicação detalhada do rationale, a lógica por trás da solução.
 - Mostra como o padrão emerge das pressões do ambiente e como equilibra essas forças.

Estrutura típica

Seção	Descrição
Contexto	Ambiente e pré-condições onde o padrão se aplica.
Forças	Interesses conflitantes ou limitações (tempo, recursos, complexidade).
Problema	Questão recorrente que surge desse contexto.
Solução	Mecanismo que resolve o problema balanceando as forças.
Rationale	Justificativa — por que essa solução é melhor que outras.
Consequências	Resultados, prós e contras observados.
Relacionamentos	Ligações com outros padrões (antes, depois, complementares).

Exemplo simplificado — Factory Method, estilo Coplien

- **Contexto:** Em sistemas com várias classes derivadas, o código cliente não deve depender de implementações concretas.
- **Forças:** Flexibilidade vs. simplicidade; custo de abstração vs. independência de implementação.
- **Problema:** Como criar objetos sem acoplar o código às classes concretas?
- **Solução:** Defina uma operação de criação em uma classe base, delegando às subclasses a responsabilidade de instanciar o tipo adequado.
- **Rationale:** Isso isola a criação da estrutura de uso, permitindo extensibilidade sem modificação direta.
- **Consequência:** Reduz acoplamento, mas aumenta o número de classes e a complexidade hierárquica.
- **Quando usar esse formato**
 - Em padrões de processo, organizacionais ou de análise.
 - Quando é importante entender as razões e trade-offs da solução, não apenas sua forma.

Comparação Geral dos Formatos

Aspecto	GoF	Alexandriano	Coplien
Foco	Implementação e estrutura de classes	Contexto e narrativa	Contexto e rationale
Linguagem	Técnica e formal	Descritiva e fluida	Analítica e reflexiva
Uso típico	Padrões de projeto	Padrões de análise e arquitetura	Padrões organizacionais e de processo
Vantagem	Clareza técnica e reutilização direta	Facilidade de leitura e entendimento humano	Explicação das motivações e forças
Exemplo clássico	<i>Observer, Strategy, Singleton</i>	<i>Light on Two Sides of Every Room</i>	<i>Pair Programming, Architect Also Implements</i>

Linguagens e Catálogos de Padrões

- Um padrão raramente atua sozinho: eles trabalham em conjunto.
- Quando organizados e inter-relacionados, formam Linguagens de Padrões.
- Exemplo: Alexander cria “A Pattern Language” para urbanismo (1977), com padrões como Courtyard, Terraced Slope, Garden.
- **No software, temos linguagens como:**
 - Design Patterns (GoF) – OO clássico.
 - Core J2EE Patterns – Padrões para aplicações Java corporativas.
 - POSA (Pattern-Oriented Software Architecture) – Padrões arquiteturais.
 - Tropyc – Linguagem de padrões para segurança.

O que NÃO é um padrão

- “Um padrão não é uma simples regra, receita, algoritmo ou estrutura de dados.”
- Ele deve:
 - Resolver um problema recorrente;
 - Ser uma solução comprovada (regra de três usos conhecidos);
 - Ter potencial de reutilização em contextos distintos.

Padrões e o Reuso de Software

- O reuso de software evoluiu:
 - Reuso de código → funções, bibliotecas
 - Reuso de componentes → frameworks, módulos
 - Reuso de conhecimento → padrões
- Padrões são o nível mais abstrato de reuso, pois permitem replicar estruturas de pensamento e design.
- “Engenheiros não começam do zero — reutilizam ideias que funcionaram antes.”

Comunidade e Ética dos Padrões

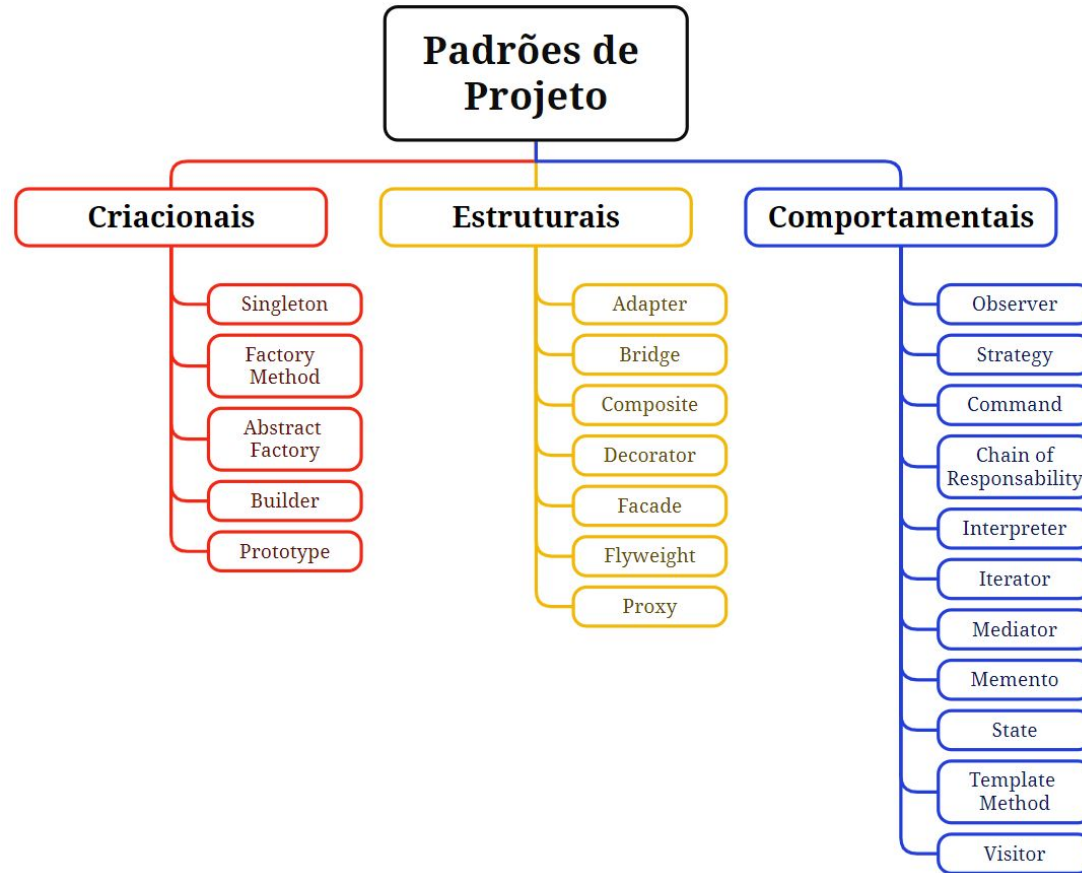
- **Comunidade Internacional:**
 - Hillside Group (EUA e Europa)
 - Conferências PLoP, EuroPLoP, SugarLoafPLoP (Brasil)
 - Repositórios como o Portland Pattern Repository (c2.com)
- **Ética de Padrões (Buschmann):**
 - Não capture apenas suas próprias ideias.
 - Dê crédito a quem criou o padrão.
 - Padrões não resolvem tudo.
 - O foco é compartilhar conhecimento, não fama.

Exemplos de Catálogos de Padrões

Catálogo	Área	Autores
Design Patterns (GoF)	OO / Engenharia de Software	Gamma et al.
POSA – Pattern-Oriented Software Architecture	Arquitetura de sistemas	Buschmann et al.
Core J2EE Patterns	Java corporativo (camadas e integração)	Alur et al.
Tropyc	Criptografia e segurança	Andrade, Viana, Bringel Filho
Organizational Patterns	Ágil e gestão de times	Coplien & Harrison

Classificação GoF

Tipo	Foco	Exemplos
Criação (Creational)	Como criar objetos	Singleton, Factory Method, Builder
Estruturais (Structural)	Como compor classes e objetos	Adapter, Composite, Decorator, Facade
Comportamentais (Behavioral)	Como objetos interagem	Observer, Strategy, Command, State



Resumo

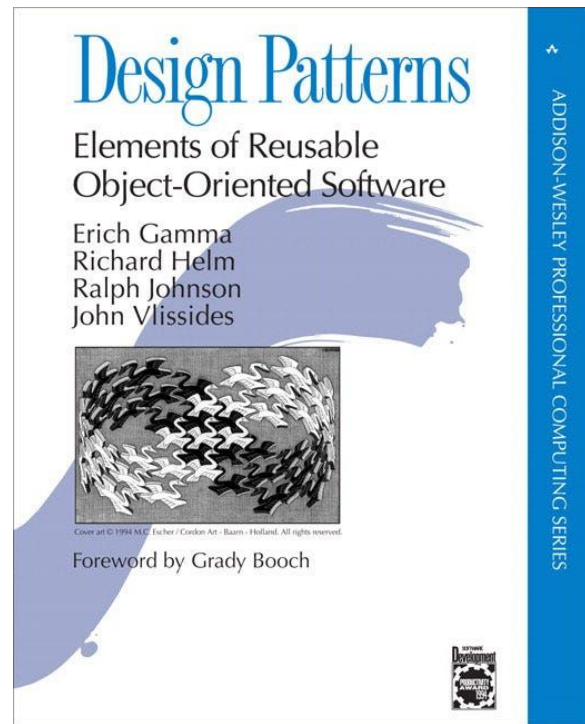
- Padrões de Software:
 - São soluções reutilizáveis para problemas recorrentes.
 - Têm três pilares: problema, contexto e solução.
 - Capturam experiência e promovem comunicação técnica.
 - Foram popularizados pelo GoF (1994).
 - Formam hoje uma linguagem comum para engenheiros e arquitetos de software.
- “Padrões são a memória coletiva da engenharia de software.”

Perguntas para debate em sala:

- Você já aplicou algum padrão sem saber?
- Quais padrões de projeto você reconhece em frameworks que usa (Android, Spring, Django...)?
- Padrões devem ser aplicados sempre ou com moderação?

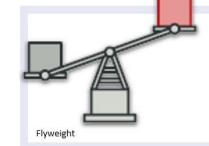
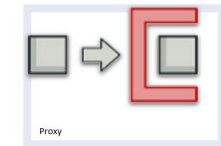
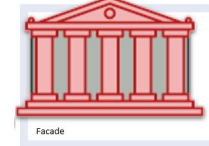
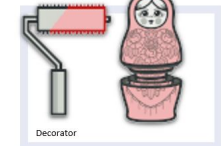
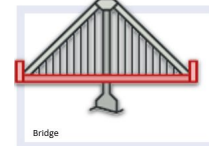
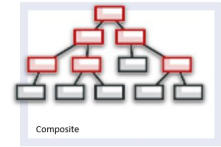
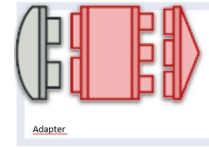
O Papel do GoF e os 23 Padrões Clássicos - Motivação

- Nos anos 1990, os sistemas orientados a objetos tornaram-se complexos, e os desenvolvedores perceberam que muitos problemas se repetiam — com soluções semelhantes, mas implementadas de formas diferentes.
- O GoF catalogou essas soluções, padronizando nomes, estruturas e intenções, criando um vocabulário universal de design.
- **Objetivo do livro GoF:** “Fornecer soluções comprovadas para problemas recorrentes de design OO, promovendo reuso e comunicação entre desenvolvedores.”



Classificação GoF — A Essência dos 23 Padrões

- Os autores do livro “Design Patterns: Elements of Reusable Object-Oriented Software” (Gamma et al., 1994) classificaram os padrões segundo dois eixos:
 - Propósito (Purpose)** → o que o padrão faz (criação, estrutura ou comportamento).
 - Escopo (Scope)** → se ele atua no nível de classe (estático, via herança) ou nível de objeto (dinâmico, via composição).
- O mais conhecido é o primeiro eixo — o propósito, que define as três famílias.



Padrões Creacionais (Criação)

- **Foco:** Como criar objetos de maneira flexível e controlada.
- **Problema que resolvem:** Evitar o acoplamento direto ao operador new e permitir que o sistema escolha quais classes instanciar, quando e como. Esses padrões escondem os detalhes da criação, tornando o código mais genérico e desacoplado.
- **Motivação**
 - Facilitar a troca de implementações sem alterar o código cliente.
 - Adiar a criação até o momento adequado.
 - Controlar o número de instâncias (ex: Singleton).

Exemplos clássicos

Padrão	Ideia principal	Analogia
Singleton	Garante que exista apenas uma instância global.	Um “gerente único” em uma empresa.
Factory Method	Delega a criação de objetos às subclasses.	Um “fábrica especializada” que decide o produto final.
Abstract Factory	Cria famílias de objetos relacionados.	Uma “fábrica de fábricas” (ex: tema dark/light).
Builder	Separa construção de representação.	Montar um carro peça por peça.
Prototype	Clona objetos existentes em vez de criar do zero.	Copiar um modelo-base já configurado.

Benefício central: Encapsular a lógica de criação, permitindo construir objetos complexos sem acoplamento direto ao código cliente.

Padrões Estruturais

- **Foco:** Como compor classes e objetos para formar estruturas maiores.
- **Problema que resolvem:** Facilitar a montagem de componentes e a reutilização de código, garantindo que as partes do sistema possam ser conectadas ou substituídas de forma transparente.
- **Motivação**
 - Promover composição sobre herança.
 - Permitir extensão dinâmica de funcionalidades.
 - Criar interfaces unificadas para sistemas complexos.

Exemplos clássicos

Padrão	Ideia principal	Analogia
Adapter	Converte uma interface em outra esperada.	Um adaptador de tomada que faz um plug funcionar.
Composite	Permite tratar objetos individuais e compostos da mesma forma.	Uma árvore de diretórios (pasta contém pastas e arquivos).
Decorator	Adiciona responsabilidades dinamicamente.	Envelopar um presente com várias camadas de papel.
Facade	Cria uma interface simplificada para um subsistema complexo.	Um recepcionista que intermedia o contato com vários departamentos.
Proxy	Controla o acesso a outro objeto (ex: remoto ou caro de instanciar).	Um procurador agindo em nome de alguém.
Bridge	Separa abstração da implementação.	Um controle remoto que opera TVs diferentes.
Flyweight	Compartilha objetos leves para economizar memória.	Letras de um editor de texto — cada letra é reutilizada.

Benefício central: Organizar a estrutura do sistema, permitindo que as partes se encaixem de forma flexível, compondo sistemas extensíveis e de fácil manutenção.

Padrões Comportamentais

- **Foco:** Como objetos interagem e se comunicam entre si.
- **Problema que resolvem:** Lidar com a coordenação de responsabilidades, a comunicação entre objetos e o fluxo de controle dentro do sistema. Eles descrevem como distribuir e desacoplar comportamentos complexos.
- **Motivação**
 - Evitar dependências rígidas entre objetos comunicantes.
 - Tornar o comportamento configurável ou intercambiável.
 - Encapsular regras e protocolos de comunicação.

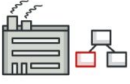
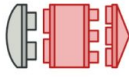
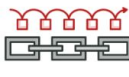
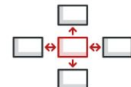
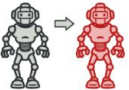
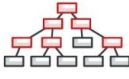
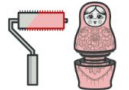
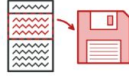
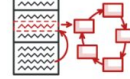
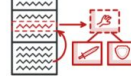
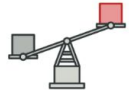
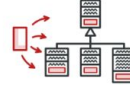
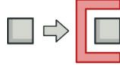
Exemplos clássicos

Padrão	Ideia principal	Analogia
Observer	Um objeto notifica automaticamente seus dependentes.	Inscrever-se para receber alertas de um canal.
Strategy	Permite trocar o algoritmo em tempo de execução.	Escolher a melhor rota no GPS conforme o trânsito.
Command	Encapsula uma solicitação como objeto.	Um controle remoto que envia comandos à TV.
State	Altera o comportamento conforme o estado interno.	Um semáforo que muda de cor e comportamento.
Mediator	Centraliza a comunicação entre objetos.	Um coordenador de grupo que evita confusões diretas.
Iterator	Percorre elementos sem expor estrutura interna.	Um cursor que percorre itens de uma lista.
Chain of Responsibility	Vários objetos tentam tratar uma requisição em cadeia.	Suporte técnico passando um chamado até alguém resolver.
Template Method	Define o esqueleto de um algoritmo e deixa subclasses preencherem detalhes.	Receita de bolo com etapas fixas e variações.
Memento	Permite salvar e restaurar o estado de um objeto.	Função “desfazer” (Undo).

Benefício central: Organizar a interação entre objetos de forma que o sistema se mantenha flexível, extensível e de fácil evolução.

Analogia

Tipo	Metáfora
Creacional	“A Fábrica” — criação dos objetos.
Estrutural	“O Lego” — montagem de peças.
Comportamental	“O Teatro” — objetos atuando juntos.

							
Factory Method	Abstract Factory	Adapter	Bridge	Chain of Responsibility	Command	Iterator	Mediator
							
Builder	Prototype	Composite	Decorator	Memento	Observer	State	Strategy
							
Singleton		Facade	Flyweight	Template Method	Visitor		
							
		Proxy					

Benefícios dos Padrões GoF

Benefício	Descrição
Reuso	Soluções testadas e validadas.
Padronização	Linguagem comum entre desenvolvedores.
Desacoplamento	Classes menos dependentes entre si.
Facilidade de manutenção	Mudanças localizadas e controladas.
Escalabilidade	Estruturas mais flexíveis e expansíveis.

“Design patterns tornam o design de software mais robusto, comunicável e compreensível.”

Cuidados e Limitações

- Não são receitas prontas, mas modelos de pensamento.
- Podem aumentar complexidade se aplicados indevidamente.
- Devem ser usados com consciência, não por modismo.
- Nem todo problema precisa de um padrão formal — às vezes, um bom design simples já basta.

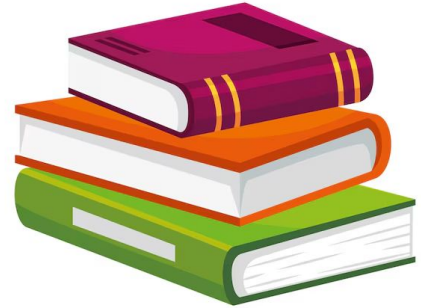
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

